

AIAM Technology LLC

Website Project Documentation

AIAM Technology LLC

May 30, 2026

Formal website preview — full launch planned for June 2026

Contents

1	Introduction	3
1.1	Project Goals	3
1.2	Repository	3
2	Technology Stack	3
3	Project Structure	3
3.1	File Responsibilities	4
4	Site Architecture and Navigation	4
4.1	Single-Page, Multi-View Model	4
4.2	Page Content Overview	4
4.3	Shared Footer	4
5	Responsive Design and Layout Behavior	5
5.1	Breakpoints	5
5.2	Fit-to-Viewport Mode (Large Screens)	5
6	JavaScript Behavior (main.js)	5
6.1	Mobile Navigation	5
6.2	View Routing	5
6.3	Contact Form Submission	6
7	Hosting on Netlify	6
7.1	Why <code>index.html</code> Is at the Repository Root	6
7.2	Recommended Netlify Configuration	6
7.3	Deployment Workflow (Git Integration)	6
7.4	Optional: <code>netlify.toml</code>	6
7.5	Local Preview Before Deploy	7
7.6	Netlify Considerations for This Project	7
8	Form Collection with Formspree	7
8.1	Endpoint	7
8.2	Form Fields	8
8.3	Submission Method (AJAX via Fetch)	8
8.4	User Feedback	8
8.5	Formspree Account Setup	8
8.6	Testing Checklist	9
8.7	Security and Privacy Notes	9
9	Maintenance and Future Work	9
9.1	Routine Updates	9
9.2	Planned for Formal Launch (June 2026)	9
9.3	Troubleshooting	9
10	Conclusion	10

1 Introduction

This document describes the AIAM Technology LLC marketing website: its purpose, architecture, source layout, runtime behavior, deployment on **Netlify**, and contact-form handling through **Formspree**.

The site is a static, client-side web application built with HTML, CSS, and JavaScript. It presents four primary sections (Home, About, Technology, and Contact) within a single page, using hash-based view switching so navigation feels like separate tabs without full page reloads.

1.1 Project Goals

- Present AIAM’s machine-side automation capabilities to manufacturers.
- Provide a professional landing experience aligned with the approved visual mockup.
- Offer a working contact channel before the formal website launch.
- Deploy easily as static files on Netlify with Git-based continuous deployment.

1.2 Repository

- **GitHub:** <https://github.com/ASilvaGSW/AIAM.git>
- **Default branch:** main
- **Contact email (public):** kenjin@aiam-tech.com
- **Operating region:** Mission, Texas / Reynosa, Mexico

2 Technology Stack

Layer	Technology
Markup	HTML5 (semantic sections, ARIA labels)
Styling	CSS3 (Flexbox, Grid, custom properties, media queries)
Scripting	Vanilla JavaScript (ES5-compatible IIFE)
Typography	Google Fonts — Inter
Hosting	Netlify (static site hosting)
Form backend	Formspree (https://formspree.io)
Version control	Git / GitHub

Table 1: Technology stack summary

No build step, package manager, or server-side runtime is required. The site can be opened locally by serving the repository root or by opening `index.html` through a local HTTP server (recommended for Formspree testing).

3 Project Structure

The publish root for Netlify is the repository root. `index.html` must remain at the top level so Netlify serves it as the site entry point.

```
AIAM/
|-- index.html      # Site entry point and all page views
|-- css/
|   '-- styles.css  # Global layout, components, responsive rules
|-- js/
|   '-- main.js     # Navigation, view switching, Formspree submit
|-- src/
```

```
| '-- img/           # Brand and content images
|   |-- Logo_5_19_2026.png
|   |-- aiam007.png    # Original design mockup (reference)
|   |-- hero_machine.png # Hero section image
|   |-- card_wire.png   # Wire Harness card thumbnail
|   |-- card_vision.png  # AI Vision card thumbnail
|   |-- card_backup.png  # Power Backup card thumbnail
|-- docs/
|-- project-documentation.tex # This document
```

3.1 File Responsibilities

`index.html`

Defines the header, four view sections (`#view-home`, `#view-about`, `#view-technology`, `#view-contact`), the shared footer value strip, and the contact form markup.

`css/styles.css`

Implements the visual design, sticky header, hero diagonal image clip, feature cards, inner-page layouts, mobile navigation, footer stickiness, and the fit-to-viewport layout for large desktop screens.

`js/main.js`

Handles mobile menu toggle, client-side view routing via `data-view` attributes, URL hash updates, and asynchronous Formspree form submission.

`src/img/`

Static image assets. Hero and card images were derived from the approved mockup for visual consistency.

4 Site Architecture and Navigation

4.1 Single-Page, Multi-View Model

Instead of separate HTML files per section, the application uses a **view-switching** pattern:

1. Each major section is a `<section class="view">` element with a `data-view` attribute (`home`, `about`, `technology`, `contact`).
2. Only the view with class `active` is displayed.
3. Navigation links, the logo, and in-page CTAs carry matching `data-view` attributes.
4. JavaScript intercepts clicks, prevents default navigation, and toggles the active view.
5. The browser URL hash is updated (e.g. `#contact`) using `history.replaceState`, enabling direct links and refresh persistence.

4.2 Page Content Overview

4.3 Shared Footer

A dark value strip appears on every view, listing:

- Engineered for Reliability
- Modular by Design
- Built for Manufacturability
- Practical Support, Real Results

The layout uses a flex column on `body` so the footer remains at the bottom of the viewport on desktop, even when inner-page content is shorter than the screen height.

View	Content
Home	Hero with company headline, description, “Formal Website Coming June 2026” badge, tagline, contact row, hero machine image, and three feature cards (Wire Harness, AI Vision, Machine Power Backup).
About	Company mission, regional presence, tagline, and four value propositions with icons.
Technology	Expanded descriptions of the three core technology areas with larger cards.
Contact	Contact information and a message form (name, email, message) integrated with Formspre.

Table 2: View sections

5 Responsive Design and Layout Behavior

5.1 Breakpoints

- **Desktop / laptop ($\geq 961\text{px}$):** Two-column hero, three-column card grids, horizontal navigation.
- **Tablet ($\leq 960\text{px}$):** Stacked hero, single-column cards and inner-page grids.
- **Mobile ($\leq 720\text{px}$):** Hamburger menu, increased side padding, vertical card layout, full-width submit button.
- **Small mobile ($\leq 400\text{px}$):** Further reduced typography and padding.

5.2 Fit-to-Viewport Mode (Large Screens)

On viewports with `min-width: 961px` and `min-height: 740px`, additional CSS rules apply:

- `body` is locked to `100vh` with `overflow: hidden`.
- The Home view compresses vertical spacing so hero, cards, header, and footer fit without scrolling.
- Inner views may scroll internally if content exceeds available height.

On smaller or shorter screens, normal document scrolling is used instead.

6 JavaScript Behavior (`main.js`)

6.1 Mobile Navigation

The hamburger button toggles the `open` class on `#mainNav` and updates `aria-expanded` for accessibility.

6.2 View Routing

The `showView(name)` function:

- Activates the matching `.view` section.
- Updates active state on `.nav-link` elements.
- Resets scroll on the newly shown view.
- Closes the mobile menu.
- Writes the hash fragment to the URL.

On initial load, if the URL contains a valid hash (e.g. `#technology`), that view is opened automatically.

6.3 Contact Form Submission

The contact form uses the Fetch API to POST to Formspree without leaving the page. See Section 8 for backend details.

7 Hosting on Netlify

Netlify is a static hosting platform well suited to this project because the site consists entirely of HTML, CSS, JavaScript, and image assets with no server-side code.

7.1 Why `index.html` Is at the Repository Root

Netlify (and most static hosts) treat the configured **publish directory** as the web root. The file `index.html` in that directory becomes the default document for `/`. Placing `index.html` at the repository root ensures:

- Correct resolution of relative paths such as `css/styles.css`, `js/main.js`, and `src/img/...`
- No extra redirect or rewrite rules are needed for the home page.
- Asset URLs remain simple and portable.

7.2 Recommended Netlify Configuration

Setting	Value
Publish directory	<code>/</code> (repository root)
Build command	<i>(none)</i>
Production branch	<code>main</code>

Table 3: Typical Netlify site settings for this project

Because there is no build pipeline, Netlify deploys files as-is after each push to the connected Git branch.

7.3 Deployment Workflow (Git Integration)

1. Connect the Netlify site to the GitHub repository <https://github.com/ASilvaGSW/AIAM.git>.
2. Set the production branch to `main`.
3. Confirm the publish directory is the repository root.
4. Push commits to `main`; Netlify triggers an automatic deploy.
5. Review the deploy log in the Netlify dashboard if a deployment fails.
6. Optionally assign a custom domain (e.g. `aiam-tech.com`) under **Domain management** and configure DNS with Netlify or an external registrar.

7.4 Optional: `netlify.toml`

Although not required, a minimal `netlify.toml` at the repository root can document deployment settings explicitly:

```
[build]
  publish = "."
  command = ""

[[headers]]
  for = "/*"
  [headers.values]
    X-Frame-Options = "DENY"
    X-Content-Type-Options = "nosniff"
    Referrer-Policy = "strict-origin-when-cross-origin"
```

This file is optional; equivalent settings can be configured entirely in the Netlify UI.

7.5 Local Preview Before Deploy

To approximate the Netlify environment locally:

```
# Using Python (if installed)
python -m http.server 8080

# Or using Node npx
npx serve .
```

Then open <http://localhost:8080>. Use HTTP (not file://) when testing Formspree submissions.

7.6 Netlify Considerations for This Project

- **HTTPS:** Netlify provides free TLS certificates; Formspree works over HTTPS in production.
- **SPA-style hashes:** Hash routes (`#about`, etc.) do not require special Netlify redirects because routing is handled client-side.
- **Form submissions:** Forms are sent directly from the browser to Formspree; Netlify does not process form data unless Netlify Forms is enabled separately (not used in this project).
- **Caching:** After CSS/JS updates, users may need a hard refresh (`+Shift+R`) if assets appear stale due to browser cache.

8 Form Collection with Formspree

Formspree is a third-party service that receives HTML form submissions and forwards them by email without requiring a custom backend.

8.1 Endpoint

The contact form posts to:

<https://formspree.io/f/xvzyywkz>

This endpoint is configured in `index.html`:

```
<form class="contact-form" id="contactForm"
  action="https://formspree.io/f/xvzyywkz"
  method="POST" novalidate>
```

8.2 Form Fields

Field name	Input type	Required
name	text	Yes
email	email	Yes
message	textarea	Yes

Table 4: Contact form fields sent to Formspree

Field names must match Formspree expectations. The `email` field is used by Formspree for reply-to addressing and validation.

8.3 Submission Method (AJAX via Fetch)

Rather than a traditional full-page POST, `main.js` intercepts submit events and sends data asynchronously:

```
fetch(form.action, {
  method: "POST",
  body: new FormData(form),
  headers: { Accept: "application/json" }
})
```

Benefits of this approach:

- The user stays on the Contact view (consistent with SPA navigation).
- Success and error messages appear inline below the submit button.
- The submit button shows a “Sending...” state and is disabled during the request.

8.4 User Feedback

- **Success:** “Thanks — your message has been sent. We’ll get back to you soon.” The form is reset.
- **Validation error:** Browser-native validation via `checkValidity()` before submit.
- **Formspree error:** Server error messages are displayed when Formspree returns JSON error details.
- **Network error:** User is prompted to retry or email directly.

8.5 Formspree Account Setup

1. Create or sign in to a Formspree account at <https://formspree.io>.
2. Create a form and note the form ID (`xvzyywkz` in this project).
3. Set the notification email address where submissions should be delivered.
4. On the **first real submission**, Formspree typically sends a confirmation email to verify the form endpoint — approve it to activate delivery.
5. Optionally enable spam filtering (reCAPTCHA, honeypot) in the Formspree dashboard for production hardening.

8.6 Testing Checklist

- Serve the site over HTTP/HTTPS (not `file://`).
- Submit a test message from the Contact view.
- Confirm receipt in the Formspree dashboard and notification inbox.
- Test invalid email and empty fields to verify client-side validation.
- Test from the live Netlify URL after deployment.

8.7 Security and Privacy Notes

- The Formspree endpoint URL is public in client-side code; this is normal for static sites. Abuse mitigation should be configured in Formspree.
- Do not embed API secrets in frontend code; only the public form endpoint is used.
- Consider adding a privacy notice if required by jurisdiction or company policy before the formal launch.

9 Maintenance and Future Work

9.1 Routine Updates

- Edit content in `index.html` for copy changes.
- Edit `css/styles.css` for layout and visual updates.
- Replace images in `src/img/` keeping filenames or updating HTML references accordingly.
- Commit and push to `main` to trigger Netlify redeployment.

9.2 Planned for Formal Launch (June 2026)

- Expanded content and dedicated pages if moving beyond single-page views.
- Custom domain and production SEO metadata.
- Analytics integration (e.g. Netlify Analytics, Plausible, or GA4).
- Enhanced Formspree configuration (spam protection, autoresponders).
- Removal of “Formal Website Coming June 2026” preview messaging.

9.3 Troubleshooting

Styles or scripts not loading on Netlify

Verify `index.html` is at the publish root and paths use `css/...` and `js/...` without `../`.

Form submissions fail locally

Use a local HTTP server; Formspree may reject requests from `file://` origins.

Form works locally but not on Netlify

Confirm the Formspree form is activated and the site URL is allowed if domain restrictions are enabled in Formspree.

Footer not at bottom on desktop

Ensure body flex layout rules in `styles.css` are intact and not overridden by custom edits.

Mobile layout too tight

Check `@media (max-width: 720px)` rules; container horizontal padding is controlled by `.container`.

10 Conclusion

The AIAM Technology LLC website is a lightweight static application optimized for Netlify deployment and Formspree-powered contact collection. Its structure keeps operational complexity low while supporting a polished multi-section experience on desktop and mobile. This document should serve as the primary reference for developers maintaining, deploying, and extending the project through the June 2026 formal launch.